

NATIONAL REAL-TIME PAYMENTS SYSTEM

Technical Architecture & Business Analysis Report

Prepared for:	Emily Watson, IT Technical Lead
Organization:	UK Technology Practice
Prepared by:	Architecture Team — NRTPS
Document Type:	Formal Technical Business Report
Classification:	Confidential
Version:	1.0
Date:	March 2026

Table of Contents

1. Executive Summary	4
2. Context and Scope	5
2.1 System Users and Scale	5
2.2 Supported User Segments	5
2.3 Supported Platforms	5
3. Systems and Subsystems	6
3.1 Client Interface Layer	6
3.2 API Gateway and Security Layer	6
3.3 Core Microservices	6
3.4 Banking Integration and Settlement Layer	7
3.5 Data and Infrastructure Layer	7
4. Information Exchange Between Systems	8
5. Failure Analysis	9
5.1 Technical Failures	9
5.2 Integration Failures	9
5.3 External Dependency Failures	9
6. Mitigation Strategies	10
6.1 Redundancy and High Availability	10
6.2 Circuit Breakers and Bulkheads	10
6.3 Idempotency and Exactly Once Processing	10
6.4 Horizontal Auto-Scaling	10
6.5 Real-Time Monitoring and Alerting	10
7. Contingency Plans	12
7.1 Deferred Settlement	12
7.2 Manual Reconciliation	12
7.3 User Communication Strategy	12
8. System Behavior Under Varying Load	13

8.1 Normal Daily Load	13
8.2 Peak National Events	13
9. Architectural Trade-offs	14
9.1 Consistency versus Availability	14
9.2 Cost versus Reliability	15
9.3 Real-time versus Eventual Processing	15
9.4 Security Depth versus User Experience	15
10. Global Benchmarks and Unique Selling Propositions	16
10.1 Comparative Analysis	16
10.2 Recommended Unique Selling Propositions and ROI	16
10.2.1 AI-Powered Inline Fraud Detection	16
10.2.2 Open API Marketplace	17
10.2.3 Cross-Border Payment Readiness	17
10.2.4 Digital Financial Inclusion	17
11. Conclusion	18

1. Executive Summary

This report provides a technical and business analysis of the National Real-Time Payments System (NRTPS), a digital payments platform built to replace cash and slow bank transfers with near-instant transactions for citizens, merchants, and financial institutions.

The system is designed for 150 million registered users, with 40 million daily active users, and the ability to handle over 50,000 transactions per second (TPS) during peak periods. It targets 99.999% availability, which means less than five minutes of unplanned downtime each year. The platform also meets PCI DSS, ISO 27001, and national banking regulatory requirements.

This document is addressed to the IT Technical Lead, and covers the following dimensions of the system's architecture:

- Main subsystems and inter-system information flows
- Potential failure scenarios and their mitigation strategies
- Contingency plans for when failures occur
- Adaptive system behavior under normal and peak conditions
- Key architectural trade-offs and the rationale for each design decision
- Global implementation benchmarks and the unique selling propositions (USPs) of the recommended solution, with projected return on investment (ROI)

The findings presented herein are grounded in the case study requirements provided, supplemented by analysis of comparable global implementations including India's Unified Payments Interface (UPI), Brazil's PIX, and the United States Federal Reserve's FedNow service.

2. Context and Scope

A nation seeks to modernize its payments landscape by deploying a nationally operated real-time payments infrastructure. The NRTPS is intended to serve as the primary backbone for all domestic peer-to-peer (P2P) and peer-to-merchant (P2M) transactions. Upon successful national deployment, the system is expected to expand into cross-border real-time payments, interfacing with partner countries banking networks and complying with international financial regulations.

2.1 System Users and Scale

The following user and scale parameters define the system's operational envelope:

Parameter	Value
Total Registered Users	150 million
Daily Active Users	40 million
Peak Concurrent Users	5 million
Peak Transactions Per Second	50,000+
Transaction Latency (End-to-End)	< 2 seconds
System Availability Target	99.999%

2.2 Supported User Segments

- Individual citizens: performing P2P payments and receiving funds
- Merchants (online and offline) accept QR-based P2M payments
- Banks and financial institutions: settlement, clearing, and reconciliation
- Regulators and auditors: compliance monitoring and regulatory reporting

2.3 Supported Platforms

- Mobile applications (iOS and Android)
- Web applications
- Merchant point-of-sale (POS) terminals
- Third-party integrations via REST/API interfaces

3. Systems and Subsystems

The NRTPS is architected as a layered, microservices platform. Each layer encapsulates a distinct functional concern, enabling independent scaling, deployment, and fault isolation. The principal systems and subsystems are described below.

3.1 Client Interface Layer

This layer provides all user interaction surfaces. It abstracts the underlying platform complexity and presents a consistent experience across channels.

- Mobile Applications (iOS/Android): Native apps supporting biometric authentication, QR code scanning, and push notifications.
- Web Portal: Browser-based interface for individual users and merchants.
- Merchant POS Integration: SDK or hardware-level integration for point-of-sale terminals.
- Third-Party API Gateway: RESTful APIs enabling fintech's and developers to embed payment capabilities.

3.2 API Gateway and Security Layer

All inbound traffic traverses a centralized API gateway responsible for authentication, authorization, rate limiting, and request routing.

- OAuth 2.0 / OpenID Connect: Federated identity and delegated authorization.
- Transport Layer Security (TLS 1.3): Encryption of all traffic in transit.
- Rate Limiting and DDoS Protection: Protection against volumetric attacks and API abuse.
- Load Balancing: Distributes requests across microservice instances.

3.3 Core Microservices

The business logic of the platform is decomposed into independently deployable microservices, each owning its domain data:

- User and KYC Service: Manages customer onboarding, identity verification, and account linkage to one or more bank accounts.
- Payment Orchestration Engine: Coordinates the end-to-end transaction lifecycle — initiation, validation, authorization, and confirmation.
- Fraud Detection and Risk Scoring Service: Executes machine-learning models in real time to assess transaction risk without blocking payment flow.
- Notification Service: Delivers SMS, push notification, and email confirmations upon transaction events.
- Merchant Service: Handles merchant registration, QR code generation, dashboard access, and daily settlement reporting.

- Audit and Compliance Service: Maintains an immutable event log for regulatory reporting and dispute resolution.

3.4 Banking Integration and Settlement Layer

This subsystem manages all interactions with the banking ecosystem, ensuring funds move reliably between institutions.

- Core Banking Interface: Adapter layer integrating with each bank's core banking system via standardized APIs (ISO 20022 message format).
- Inter-Bank Settlement Engine: Manages real-time gross settlement (RTGS) and deferred net settlement (DNS) depending on transaction type.
- Reconciliation Service: Verifies transaction records against bank confirmations and resolves discrepancies.
- Dispute Management Module: Facilitates chargeback initiation, adjudication, and resolution workflows.

3.5 Data and Infrastructure Layer

The data layer provides persistent storage, streaming, caching, and messaging infrastructure that underpins all services.

- Distributed Database (e.g., CockroachDB or Google Spanner): Globally distributed, strongly consistent storage for transactional data.
- Message Queue (Apache Kafka): Asynchronous, durable event streaming for decoupling microservices and enabling event replay.
- Distributed Cache (Redis Cluster): Sub-millisecond read access for session data, rate limits, and idempotency keys.

Observability Stack: Centralized logging, metrics, and distributed tracing (e.g., ELK Stack, Prometheus, Jaeger).

4. Information Exchange Between Systems

The quality of inter-system data exchange is as critical as the data itself. The table below summarizes the key information flows across the NRTPS, characterized by frequency, speed, volume, and consistency requirements.

Information Flow	Frequency	Speed Requirement	Volume	Consistency
Payment initiation request	Continuous	< 500 ms	Very High	Strong
Fraud risk score	Per transaction	< 100 ms (inline)	Very High	Eventual
Bank authorization signal	Per transaction	< 1 second	High	Strong
Inter-bank settlement instructions	Batch (T+0/T+1)	Near real-time	Medium	Strong
KYC / identity verification	On onboarding	< 10 seconds	Low	Strong
Transaction status notifications	Event-driven	< 2 seconds	High	Eventual
Audit log writes	Every transaction	Asynchronous	Very High	Eventual
Regulatory reporting exports	Daily/Monthly	Batch	Medium	Strong

Payment initiation and bank authorization signals demand strong consistency, a transaction must not be recorded as complete unless all parties have confirmed commitment. Fraud scoring and audit logs, by contrast, operate on eventual consistency models. They must not introduce latency into the payment path. Kafka-based event streaming ensures that these asynchronous workloads are processed reliably without coupling them to the synchronous payment flow.

5. Failure Analysis

A thorough failure analysis is essential for a system that processes financial transactions on a national scale. Failures may be grouped into three categories: technical, integration, and external dependency failures.

5.1 Technical Failures

- Database node failure: Partial or complete loss of database replicas, risking data inconsistency or unavailability.
- Microservice crash or memory exhaustion: A service instance may terminate unexpectedly under high load or following an unhandled exception.
- Network partition: Loss of connectivity between data center regions or between microservices, leading to split-brain scenarios.
- Message queue backlog overflow: Sustained high throughput may cause Kafka topics to accumulate lag beyond consumer capacity.
- Cache invalidation failure: Stale data in Redis may cause incorrect balance reads or idempotency key collisions.

5.2 Integration Failures

- Bank API timeout or error response: A downstream core banking system fails to respond within the required SLA window.
- ISO 20022 message format mismatch: Incompatible message schemas between the NRTPS settlement engine and a bank's legacy system.
- Merchant POS desynchronization: A POS terminal confirms a transaction locally, but the confirmation does not propagate to the central system.
- Third-party API client abuse: Poorly implemented API consumers sending malformed or excessive requests that degrade service quality.

5.3 External Dependency Failures

- KYC / identity provider outage: The national identity database or third-party KYC service becomes unavailable, blocking new user onboarding.
- Telecommunications network failure: SMS or push notification delivery fails, leaving users uninformed of transaction status.
- Regulatory reporting gateway unavailability: The regulator's submission portal is offline, preventing mandatory reporting within statutory deadlines.
- Foreign exchange rate feed interruption: In cross-border payment scenarios, live FX rate data may become unavailable, blocking currency conversion.

6. Mitigation Strategies

The following mitigation strategies are embedded into the NRTPS architecture to prevent or reduce the impact of the failure modes identified in Section 5.

6.1 Redundancy and High Availability

- Multi-region active-active deployment: The system runs concurrently across at least two geographically separate data centers, with automatic traffic failovers.
- Database replication with quorum writes: Transactional data is written to a quorum of database replicas before a write is acknowledged, preventing data loss on node failure.
- Kafka replication factor of 3: All event topics are replicated to ensure durability in the event of broker failure.

6.2 Circuit Breakers and Bulkheads

- Circuit breaker pattern (Hystrix / Resilience4j): Each bank integration is wrapped in a circuit breaker. If the bank API fails repeatedly, the circuit opens, preventing cascading failures across the payment engine.
- Bulkhead isolation: Thread pools and connection pools for each downstream dependency are isolated so that a failure in one bank's integration does not starve resources for other banks.

6.3 Idempotency and Exactly Once Processing

- Idempotency keys: Every payment initiation request carries a unique idempotency key, stored in Redis. Duplicate requests are detected and the original response replayed, preventing double-debits.
- Distributed saga pattern: Multi-step payment orchestration uses compensating transactions to roll back partial state in the event of mid-flow failure.

6.4 Horizontal Auto-Scaling

- Kubernetes Horizontal Pod Autoscaler (HPA): Microservice replicas scale automatically based on CPU utilization and custom Kafka consumer lag metrics.
- Pre-emptive scaling schedules: Known peak events (salary day, tax deadline, national festivals) trigger scheduled scale-out 30 minutes in advance, based on historical traffic patterns.

6.5 Real-Time Monitoring and Alerting

- Distributed tracing with Jaeger: End-to-end transaction traces are captured, enabling rapid identification of latency spikes or failure points.

- Prometheus and Grafana dashboards: Real-time metrics for TPS, error rates, queue depths, and database latencies are visualized and alerted upon breach of SLA thresholds.
- Synthetic transaction monitoring: Automated canary transactions are executed continuously to detect silent failures before users are affected.

7. Contingency Plans

Despite robust mitigation, failures will occur. The following contingency plans define the operational response when preventative measures are insufficient.

7.1 Deferred Settlement

When a bank is partially or fully unavailable, the NRTPS switches to a deferred settlement model. Payment instructions persisted in a durable Kafka queue and retried with exponential back-off once bank connectivity is restored. Users receive a 'Payment Queued' notification with an estimated completion time. Merchants are shown the queued status on their dashboards and are not required to re-initiate the transaction.

7.2 Manual Reconciliation

In the event of a system incident resulting in data inconsistency, the Audit and Compliance Service provide an immutable, append-only event log that can be replayed to reconstruct the correct state. A dedicated reconciliation operations team is empowered to execute manual reconciliation procedures using the following workflow:

- Extract the discrepant transaction window from the event log.
- Cross-reference against bank statement files obtained via secure file transfer.
- Identify over-debits, under-credits, or failed settlements.
- Issue correction entries through the settlement engine's manual override API.
- File a reconciliation report with the regulatory authority within 24 hours.

7.3 User Communication Strategy

Timely and transparent communication is paramount in preserving user trust during incidents. The communication strategy operates on three tiers:

- Tier 1: Automated in-app notifications: Transaction status updates (Pending, Failed, Completed) are pushed in real time via the Notification Service, across SMS, push, and email channels.
- Tier 2: Status page: A publicly accessible status page (e.g., status.nrtps.gov) is updated within five minutes of a P1 incident being declared.
- Tier 3: Press and regulatory communication: For incidents exceeding 30 minutes of service degradation, the communications and regulatory teams issue formal advisories to the central bank and, where appropriate, to the media.

8. System Behavior Under Varying Load

The NRTPS is explicitly designed to exhibit different operational modes under normal and peak conditions. This adaptive behavior is achieved through a combination of infrastructure elasticity, traffic management policies, and service-level degradation protocols.

8.1 Normal Daily Load

Under normal daily load, defined as up to 50,000 TPS and 40 million daily active users, the system operates in its baseline configuration:

- All microservices run at their standard replicas count (typically 3 - 5 replicas per service).
- Real-time fraud detection executes synchronously, in line with every transaction.
- Database connections operate within comfortable pool utilization (< 60%).
- Kafka consumer lag remains near zero, with messages processed within milliseconds.
- Full logging, tracing, and alerting pipelines are active.

8.2 Peak National Events

During national events, i.e. festivals, salary disbursement days, tax deadlines; transaction volume may spike to ten times the normal baseline. The system responds with the following adaptive mechanisms:

- Pre-emptive auto-scaling: Kubernetes scales microservice pods to their maximum replica limits based on scheduled triggers and real-time Kafka consumer lag.
- Traffic shaping and request queuing: The API gateway introduces bounded queuing to absorb burst traffic, applying fair-use rate limits per user to prevent monopolization.
- Fraud detection mode switching: During extreme peaks, the fraud engine may switch from synchronous scoring to an asynchronous post-transaction review model for low-risk transaction profiles, ensuring the payment path remains within the 2 second SLA.
- Read replica promotion: Database read operations are distributed across multiple read replicas to reduce primary database load.
- Reduced logging verbosity: Non-critical log levels are suppressed to reduce I/O overhead, with full logging reinstated once load normalizes.

9. Architectural Trade-offs

Every architectural decision involves trade-offs between competing concerns. The following analysis documents the key trade-offs made in the design of the NRTPS, articulating the rationale for each decision within the context of the system's requirements.

Trade-off Dimension	Option A	Option B	NRTPS Design Decision
Consistency vs. Availability	Strong consistency (ACID)	High availability (BASE)	Availability first for payment flow; strong consistency enforced only at commit boundaries via distributed sagas.
Cost vs. Reliability	Single-region deployment	Multi-region active-active	Premium infrastructure investment justified by 99.999% SLA; downtime cost at national scale vastly exceeds infrastructure cost.
Real-time vs. Eventual Processing	All processing synchronous	Async eventual consistency	Payment confirmation is synchronous; fraud analysis, audit logging, and settlement reconciliation are asynchronous.
Security Depth vs. User Experience	Maximum friction (step-up auth always)	Frictionless (no MFA)	Risk-based adaptive authentication: step-up MFA triggered only for high-value or anomalous transactions.

9.1 Consistency versus Availability

In accordance with the CAP theorem, a distributed system operating across geographically separated nodes must choose, in the event of a network partition, between maintaining consistency or availability. For a payments platform, the consequences of an inconsistent debit (e.g., a double-debit or a phantom credit) are severe. However, refusing all transactions during a partition is equally unacceptable at national scale.

The NRTPS resolves this tension by adopting an availability-first posture at the API boundary, requests are accepted and queued, whilst enforcing strong consistency at the point of final commitment through distributed saga orchestration and quorum-based database writes. This approach mirrors the design of Google Spanner and is consistent with the settlement behavior of India's UPI.

9.2 Cost versus Reliability

Operating a multi-region active infrastructure carries a materially higher cost than a single-region deployment. However, for a national payments system, the cost of downtime must be assessed against the economic impact of payment unavailability at scale. With 40 million daily active users and an average transaction value that may be conservatively estimated, even one hour of outage represents an economic disruption of significant national consequence. Accordingly, the investment in multi-region redundancy is both technically justified and prudent from a governance perspective.

9.3 Real-time versus Eventual Processing

Not all operations need to occur in the synchronous path. The NRTPS distinguishes clearly between operations that must be completed before a payment is confirmed, i.e. authorization, debit reservation, and bank notification, and those that can occur asynchronously: fraud analytics enrichment, regulatory log writing, settlement batch assembly, and merchant dashboard updates. By offloading non-critical operations to Kafka-backed asynchronous consumers, the system preserves sub 2 second end-to-end latency even at peak load.

9.4 Security Depth versus User Experience

Applying maximum security controls uniformly to all transactions would introduce unacceptable friction for low-value, low-risk payments. Conversely, reducing security universally would expose the platform to elevated fraud risk. NRTPS adopts a risk-based adaptive authentication model, consistent with the Payment Services Directive 2 (PSD2) Strong Customer Authentication framework applicable in the European context. High-value transactions, unusual geolocation patterns, and first-time payees trigger step-up authentication. Routine, low-value transactions between established counterparties proceed with minimal friction.

10. Global Benchmarks and Unique Selling Propositions

The NRTPS has been designed with reference to the leading real-time payment systems deployed globally. The following comparative analysis situates the NRTPS within the international landscape and articulates the additional capabilities that differentiate it from existing implementations.

10.1 Comparative Analysis

System	Country	Launched	Peak TPS	Registered Users	Key Differentiator
UPI	India	2016	100,000+	300M+	Open interoperability across all banks and wallets
PIX	Brazil	2020	40,000+	150M+	24/7 availability; zero transaction cost to end user
FedNow	USA	2023	~10,000	Growing	Federal Reserve backing; ISO 20022 compliance
Faster Payments	UK	2008	~1,000	67M+	Pioneer of real-time retail payments; widely adopted
NRTPS (Proposed)	National	2026	50,000+	150M	AI fraud detection; cross-border ready; adaptive scaling

10.2 Recommended Unique Selling Propositions and ROI

Beyond parity with global implementations, the following capabilities are recommended as distinguishing features of the NRTPS, each accompanied by a projected impact and ROI assessment.

10.2.1 AI-Powered Inline Fraud Detection

Unlike rule-based fraud systems that operate post-settlement, the NRTPS embeds machine learning inference directly in the payment flow, scoring every transaction in under 100

milliseconds. This capability is architecturally distinct from older implementations such as Faster Payments, which relied on post-transaction fraud analysis.

**Projected
ROI**

An industry-wide average fraud loss rate of 0.05–0.10% of transaction value, reduced by an estimated 30% through AI detection, yields a projected saving of £200M or more annually at national transaction volumes.

10.2.2 Open API Marketplace

The NRTPS exposes a developer facing API marketplace, enabling fintech organizations and enterprise partners to build payment-enabled applications directly on the national infrastructure. This mirrors the developer ecosystem model that has driven UPI's extraordinary adoption in India.

**Projected
ROI**

Licensing and volume-based revenue from API marketplace participants is projected to generate £50M or more per annum within three years of launch, whilst accelerating digital financial inclusion.

10.2.3 Cross-Border Payment Readiness

The architecture incorporates currency conversion microservices, compliance adapters for international KYC/AML regimes (FATF standards), and settlement interfaces designed for interoperability with SWIFT gpi and bilateral central bank agreements. This positions the NRTPS as a future gateway to cross-border real-time payments.

**Projected
ROI**

Domestic and cross-border remittance corridors represent a market in excess of £500B annually. Capturing a 1% share through NRTPS-facilitated cross-border flows equates to £5B in transaction volume subject to processing fees.

10.2.4 Digital Financial Inclusion

By enabling payments through basic smartphones and USSD channels, the NRTPS extends participation to unbanked or underbanked citizens. This social ROI whilst not directly monetizable carries significant political and economic value, increasing GDP participation and reducing the cost of cash infrastructure for the national economy.

11. Conclusion

The National Real-Time Payments System represents a transformative investment in national financial infrastructure. The architectural decisions documented, i.e. microservices decomposition, multi-region active deployment, availability-first consistency, adaptive fraud detection, and risk-based authentication, are individually well-established patterns, but their synthesis into a coherent, nationally scaled platform constitutes a significant engineering and organizational undertaking.

The system has been designed to meet stringent non-functional requirements: sub 2 second end-to-end latency, 99.999% availability, zero data loss, and compliance with PCI DSS and ISO 27001. It has also been architected with future extensibility in mind, anticipating the complexity of international expansion, evolving regulatory obligations, and the growth of an API-driven financial services ecosystem.

Benchmarked against global peers like UPI, PIX, FedNow, and Faster Payments, the NRTPS is positioned not merely as a follower, but as a platform with genuine differentiating capabilities: AI-powered inline fraud detection, an open API marketplace, and native cross-border payment readiness. These capabilities, if executed with the rigor that a system of this criticality demands, have the potential to deliver substantial financial returns, drive digital financial inclusion, and establish the nation as a leader in the global real-time payments landscape.

It is recommended that the IT Technical team review this architecture in conjunction with the relevant infrastructure and security workstreams and engage with the lead architect to progress the detailed design of the components identified in Section 3, prioritizing the payment orchestration engine, the banking integration layer, and the fraud detection service.